M2 : Measures of Parallel Algorithms

- Analytical Modeling of Parallel Programs:
  - Sources of Overhead in Parallel Programs
  - Performance Metrics for Parallel Systems,
- The Effect of Granularity on Performance;
- Taxonomy Parallel Platforms: Models (SIMD,MIMD, SPMD)
- Communication (Shared Address Space vs. Message Passing)

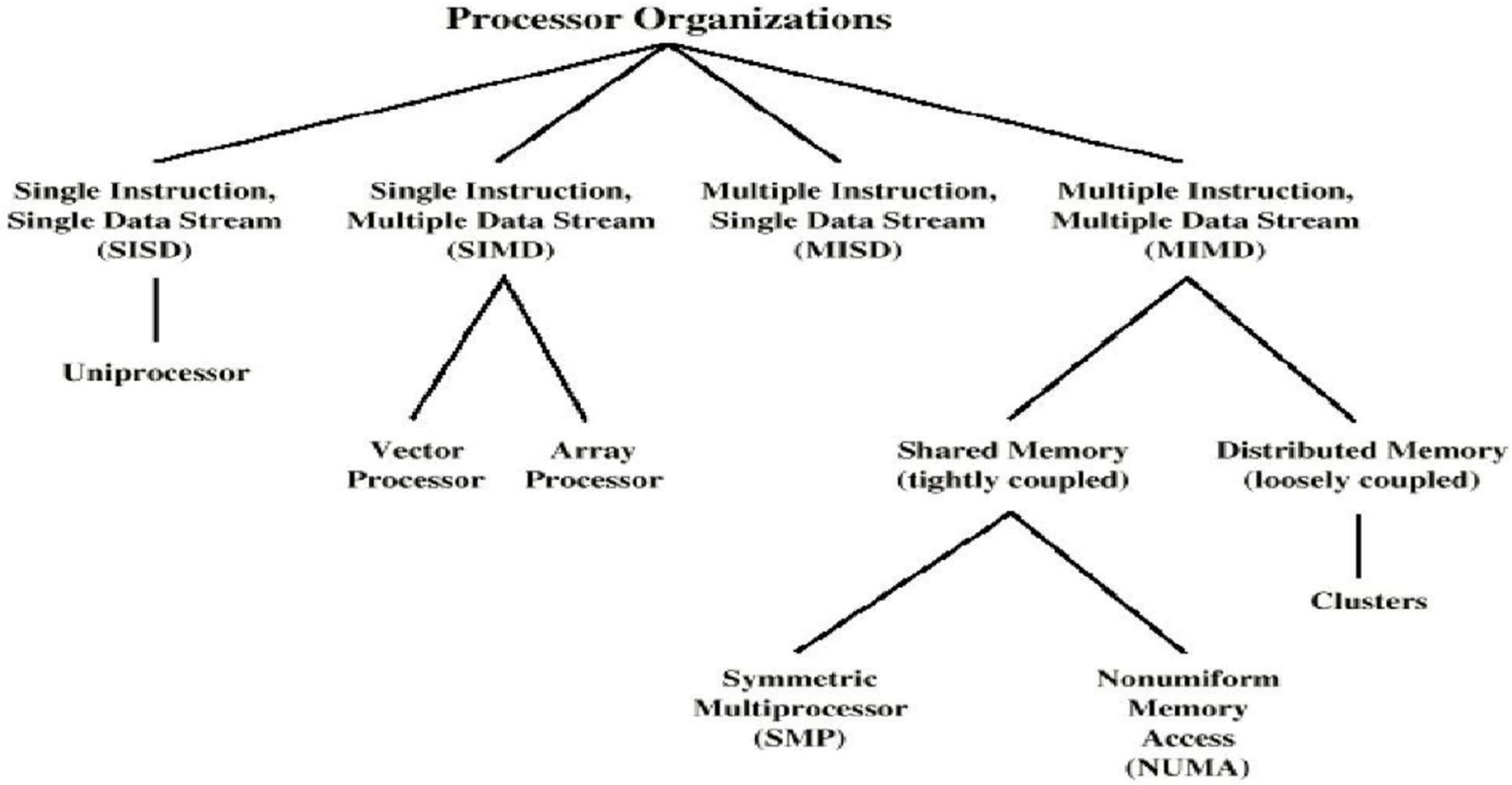
Taxonomy of Parallel computer

- SISD: single instruction and single data stream: uniprocessor
- • MISD: no commercial multiprocessor: imagine data going through a pipeline of execution engines
- • SIMD: vector architectures: lower flexibility
- • MIMD: most multiprocessors today: easy to construct withoff-the-shelf computers, most flexibility

Single Instruction / Multiple Instruction	Single Data	Multiple Data
	SISD typical CPU thread	*SIMD* vector processors GPUs
	MISD rare possibly a set of filters	*MIMD* cluster of computers multi-core processor

- MIMD machines are broadly categorized based on the way PEs are coupled to the main memory.
- **Shared-memory MIMD**
  - Tightly coupled multiprocessor systems
  - Connected to a single global memory and they all have access to it.
  - Symmetric Multi-Processing.
  - Less likely to scale
- **Distributed-memory MIMD**
  - Loosely coupled multiprocessor systems
  - All PEs have a local memory
  - Communication between PEs in this model takes place through the interconnection network.
  - _

Taxonomy of Parallel Processor Architectures

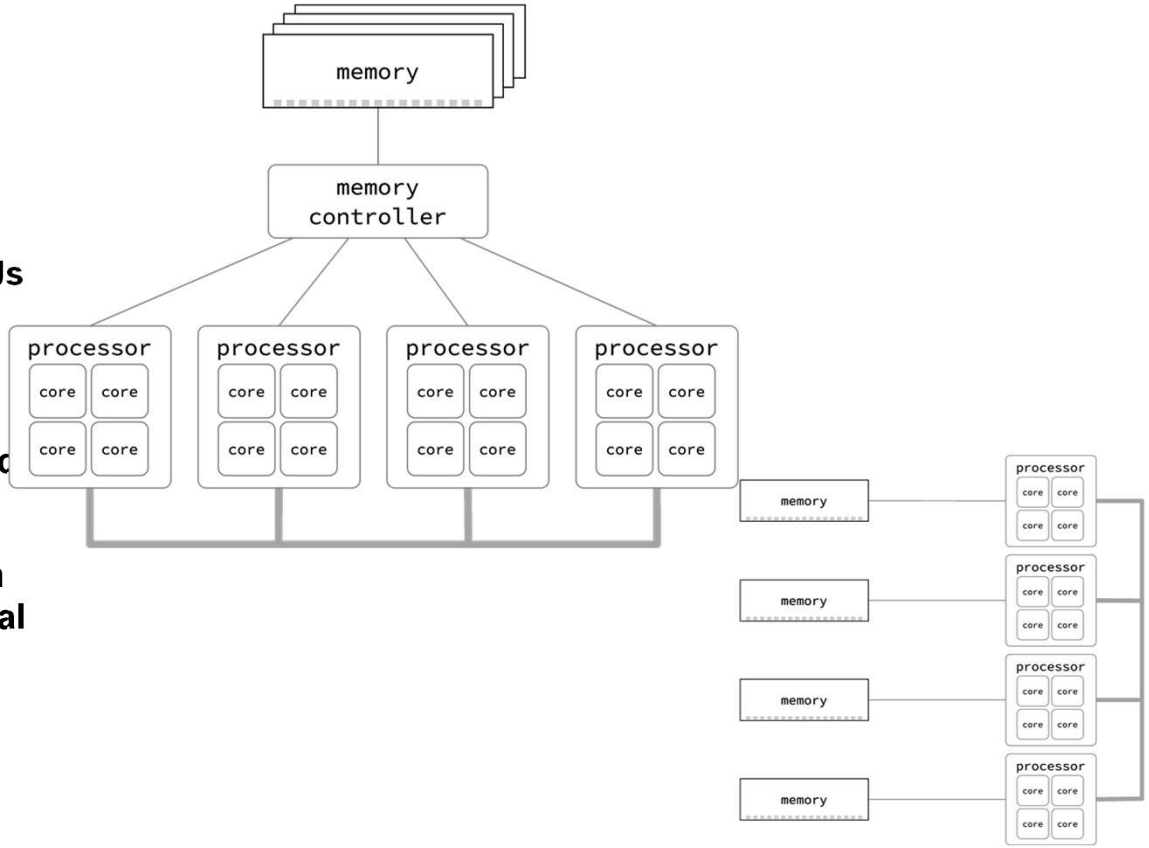


M2 : Measures of Parallel Algorithms

- Analytical Modeling of Parallel Programs:
  - Sources of Overhead in Parallel Programs
  - Performance Metrics for Parallel Systems,
- The Effect of Granularity on Performance;
- Taxonomy Parallel Platforms: Models (SIMD,MIMD, SPMD)
- Communication (Shared Address Space vs. Message Passing)

Shared Memory(UMA/NUMA)

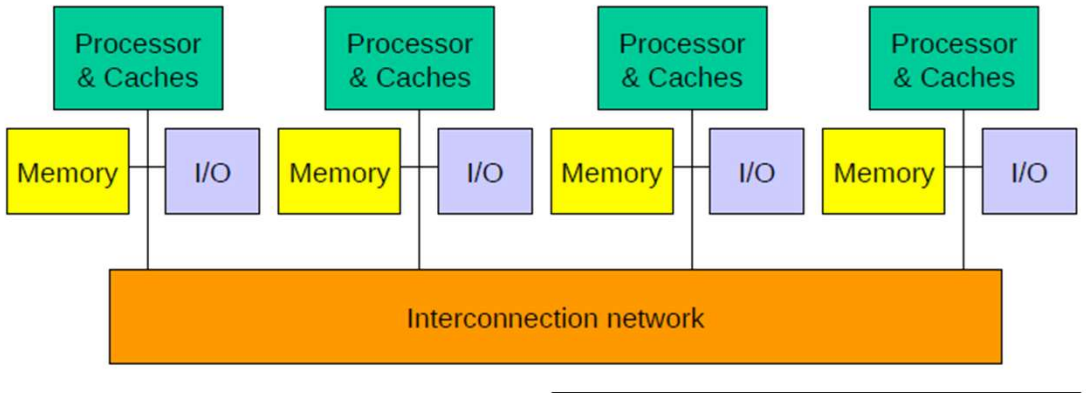
- Global address space provides a user-friendly programming perspective to memory
- Data sharing between tasks is both fast and uniform due to the proximity of memory to CPUs
- Primary disadvantage is the lack of scalability between memory and CPUs.
- coherent systems, geometrically increase traffic associated with cache/memory management.
- Programmer responsibility for synchronization constructs that insure "correct" access of global memory



Images : https://cvw.cac.cornell.edu/parallel/shared

Distributed Memory

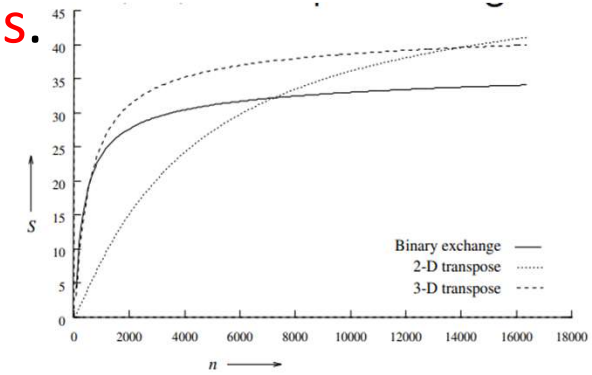
- Require a communication network to connect inter-processor memory.
  - no concept of global address space across all processors.
  - Changes it makes to its local memory have no effect on the memory of other processors.
  - Memory is scalable with number of processors.
- The programmer is responsible for many of the details associated with data communication be
- Non-uniform memory access (N



Scalability of Parallel Systems

- Programs are designed and tested for smaller problems on fewer processing elements. However, the real problems these programs are intended to solve are much larger, and the machines contain larger number of processing elements.

Scalability is a reflection of the system's ability to efficiently utilize the increased processing resource



A comparison of the speedups obtained by the binary-exchange, 2-D transpose and 3-D transpose algorithms on 64 processing elements with $t_c = 2$, $t_w = 4$, $t_s = 25$, and $t_h = 2$.

- **A parallel architecture is said to be scalable if it can be expanded (reduced) to a larger (smaller) system with a linear increase (decrease) in its performance (cost)**

A scalable system is capable of increasing its speed in proportion to the increase in the number of processors

Scaling Characteristics of Parallel Programs

- **Scaling Characteristics of Parallel Programs**
  - For a given problem size (i.e., the value of T_S remains constant), as we increase the number of processing elements, T_o increases.
  - The overall efficiency of the parallel program goes down. This is the case for all parallel programs.

$$E = \frac{S}{p} = \frac{T_S}{pT_P}$$

$$E = \frac{1}{1 + \frac{T_o}{T_S}}.$$

- The total overhead function To is an increasing function of p.

Example

Consider the problem of adding n numbers on p processing elements.

Assuming unit time for adding two numbers,

- The first phase (local summations) of the algorithm takes roughly n/p time.
- The second phase involves log p steps with a communication and an addition at each step. If a single communication takes unit time as well, the time for this phase is 2 log p.

Therefore, we can derive parallel time, speedup, and efficiency as:

$$T_P = \frac{n}{p} + 2\log p \qquad (5)$$

Speedup and Efficiency **of adding n numbers on p processing**

$$T_P = \frac{n}{p} + 2\log p \qquad (5)$$

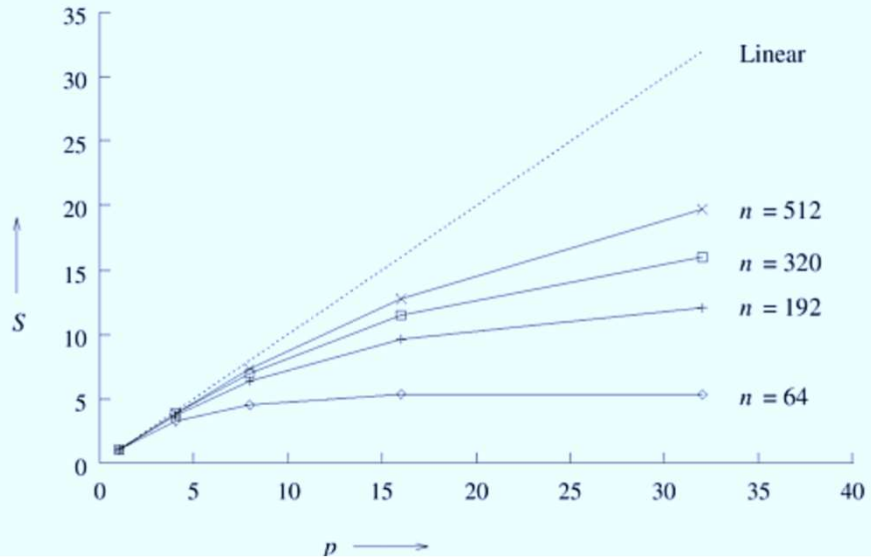
What is the speedup and efficiency

$$S = \frac{n}{\frac{n}{p} + 2\log p}$$

$$E = \frac{1}{1 + \frac{2p\log p}{n}}$$

Effect of increasing the problem size keeping the number of processing elements constant.

Figure 5.8. Speedup versus the number of processing elements for adding a list of numbers.



a given problem size, as we increase the number of processing elements, the overall efficiency of the parallel system goes down

0-1: efficiency increases if the problem size is increased keeping the number of processing elements constant

Total overhead function To is a function of both problem size TS and the number of processing elements p

Table 5.1. Efficiency as a function of *n* and *p* for adding *n* numbers on *p* processing elements.

n	*p* = 1	*p* = 4	*p* = 8	*p* = 16	*p* = 32
64	1.0	***0.80***	0.57	0.33	0.17
192	1.0	0.92	***0.80***	0.60	0.38
320	1.0	0.95	0.87	0.71	0.50
512	1.0	0.97	0.91	***0.80***	0.62

To maintain efficiency at a fixed value by simultaneously increasing the number of processing elements and the size of the problem is exhibited by many parallel systems. We call such systems scalable parallel systems

If p ↑ and n fixed → efficiency ↓ (Amdahl's Law effect).

If n ↑ and p fixed → efficiency ↑

Scalable parallel system: when increasing both n and p proportionally keeps efficiency constant.

Observation on Scaling Characteristics of Parallel Programs

- For a given problem size, as we increase the number of processing elements, the **overall efficiency of the parallel system goes down**. This phenomenon is common to all parallel systems.
- In many cases, the efficiency of a parallel system increases if the problem size is increased while keeping the number of processing elements constant.
- **Scalability of a parallel system is a measure of its capacity to increase speedup in proportion to the number of processing elements**

HPC VS. SUPERCOMPUTERS

- **Multiple supercomputers to process complex and large calculations**
  - **HPC solutions is the supercomputer**
  - **A supercomputer contains thousands of compute nodes that work together to complete one or more tasks.**

T:0: Basic Performance Improvement

Objective : **Use of tools for understanding program behavior and Performance.**

- In a large program, typically a small fragment of the code consumes most of the CPU time and/or memory, We need to identify such code fragments and need Improve the performance of such code fragments.

T0-Q1: Identify the execution (time) efficiency of a sorting program.

T0-Q2: Identify hot spots for quick sort algorithm to sort 1 million random integer using GPROF.

T0-Q3: What are the solution to improve the Performance of Performance of programing.

- # Timing a Program:
  - Run a tool to time program execution
    - E.g., Unix **`time`** command

      - Real: Wall-clock time between program invocation and termination
      - • User: CPU time spent executing the program
      - • System: CPU time spent within the OS on the program's behalf

```
$ time sort < bigfile.txt > output.txt
real    0m12.977s
user    0m12.860s
sys     0m0.010s
```

Timing Parts of a Program:

Call a function to **compute wall-clock time** consumed

- E.g., Unix `gettimeofday()` function (time since Jan 1, 1970)

```
#include <sys/time.h>

struct timeval startTime;
struct timeval endTime;
double wallClockSecondsConsumed;

gettimeofday(&startTime, NULL);
<execute some code here>
gettimeofday(&endTime, NULL);
wallClockSecondsConsumed =
   endTime.tv_sec - startTime.tv_sec +
   1.0E-6 * (endTime.tv_usec - startTime.tv_usec);
```

Call a function to compute CPU time consumed

- E.g. `clock()` function

```
#include <time.h>

clock_t startClock;
clock_t endClock;
double cpuSecondsConsumed;

startClock = clock();
<execute some code here>
endClock = clock();
cpuSecondsConsumed =
  ((double)(endClock - startClock)) / CLOCKS_PER_SEC;
```

Identifying Hot Spots

- statistics about your program's execution
  - How much time did execution of a particular function take?
  - How many times was a particular function called?
  - How many times was a particular line of code executed?
  - Which lines of code used the most time.
- How?
  - Use an execution profiler
  - • Example: `gprof` (GNU Performance Profiler)

Example program for GPROF analysis

- Sort an array of 10 million random integers
- Artificial: consumes much CPU time, generates no output

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

enum {MAX_SIZE = 10000000};
int a[MAX_SIZE]; /* Too big to fit in stack! */

void fillArray(int a[], int size)
{  int i;
   for (i = 0; i < size; i++)
      a[i] = rand();
}

void swap(int a[], int i, int j)
{   int temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}
```

```
int partition(int a[], int left, int right)
{  int first = left-1;
   int last = right;
   for (;;)
   {  while (a[++first] < a[right])
         ;
      while (a[right] < a[--last])
         if (last == left)
            break;
      if (first >= last)
         break;
      swap(a, first, last);
   }
   swap(a, first, right);
   return first;
}
```

https://bheemhh.github.io/#/HPC26

```
…
void quicksort(int a[], int left, int right)
{  if (right > left)
   {  int mid = partition(a, left, right);
      quicksort(a, left, mid - 1);
      quicksort(a, mid + 1, right);
   }
}

int main(void)
{  fillArray(a, MAX_SIZE);
   quicksort(a, 0, MAX_SIZE - 1);
   return 0;
}
```

Step 1: Instrument the program

`gcc217 –pg mysort.c –o mysort`

- Adds profiling code to mysort, that is...
- "Instruments" `mysort`

Step 2: Run the program

`mysort`

- Creates file `gmon.out` containing statistics

Step 3: Create a report

`gprof mysort > myreport`

Uses `mysort` and `gmon.out` to create textual report

Step 4: Examine the report

`cat myreport`

The GPROF Report

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
84.54	2.27	2.27	6665307	0.00	0.00	partition
9.33	2.53	0.25	54328749	0.00	0.00	swap
2.99	2.61	0.08	1	0.08	2.61	quicksort
2.61	2.68	0.07	1	0.07	0.07	fillArray

Each line describes one function

- **name**: name of the function
- **%time**: percentage of time spent executing this function
- **cumulative seconds**: [skipping, as this isn't all that useful]
- **self seconds**: time spent executing this function
- **calls**: number of times function was called (excluding recursive)
- **self s/call**: average time per execution (excluding descendents)
- **total s/call**: average time per execution (including descendents)

```
/* Program to demonstrate time taken by function fun() */
#include <stdio.h>
#include <time.h>

// A function that terminates when enter key is pressed
void fun()
{
    printf("fun() starts \n");
    printf("Press enter to stop fun \n");
    while(1)
    {
        if (getchar())
            break;
    }
    printf("fun() ends \n");
}

// The main program calls fun() and measures time taken by fun()
int main()
{
    // Calculate the time taken by fun()
    clock_t t;
    t = clock();
    fun();
    t = clock() - t;
    double time_taken = ((double)t)/CLOCKS_PER_SEC; // in seconds

    printf("fun() took %f seconds to execute \n", time_taken);
    return 0;
}
```

Overall speedup if we make 90% of a program run 10 times faster.

$F = 0.9 \quad S = 10$

$$\text{Overall Speedup} = \frac{1}{(1-0.9)+\frac{0.9}{10}} = \frac{1}{0.1+0.09} = 5.26$$

Overall speedup if we make 80% of a program run 20% faster.

$F = 0.8 \quad S = 1.2$

$$\text{Overall Speedup} = \frac{1}{(1-0.8)+\frac{0.8}{1.2}} = \frac{1}{0.2+0.66} = 1.153$$